COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/08/2026	**DATE SUBMITTED:**	08/09/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 7.1 **[Sorting Algorithms]**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

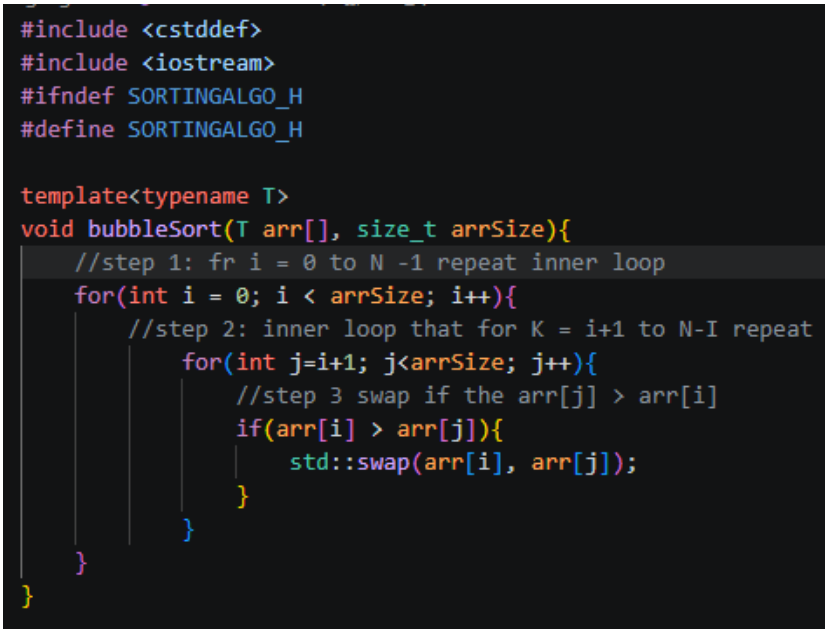
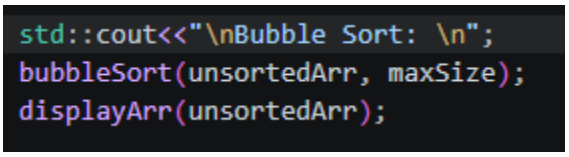
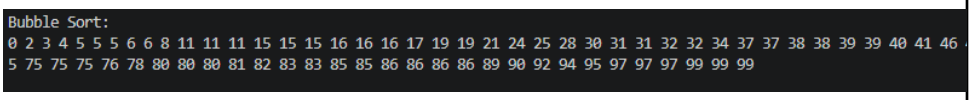
int main () {
    //create an array with unsorted list
    int unsortedArr[maxSize] = {};

    //use time (0) as the seed
    srand(time(0));
    //populate the array
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
    //display the randomly generated array
    std::cout<<"Unsorted Arr: \n";
    displayArr(unsortedArr);
```

```
PS C:\Users\Benedict\Documents\HOA7p1> & C:\Users\Benedict\Documents\HOA7p1\main.exe
Unsorted Arr:
38 66 39 85 30 59 67 38 2 90 92 0 21 61 99 75 94 95 15 86 68 58 58 40 97 74 11 4 81 32 56 61 83
1 97 5 75 16 16 65 64 58 78 34 31 80 15 52 8 73 19 53 86 65 3 19 24 41 80 99 82 68 76 70
```

Observations

This has a time complexity with the worst and best case scenario of **O(1)** because the size of the array is strictly hardcoded to loop to 100 elements, the program will always perform the exact same number of operations every single time it runs.

Code + Console Screenshot	  
Observations	This function shows a time complexity with the worst case scenario of **O(N^2)** and a best case scenario of **O(N^2)** too because there's no way to break out early because even an already sorted array still runs the full loop.

Code + Console Screenshot

```
template <typename T>
//array, current index, size of array
int routineSmallest(T arr[], int K, const int arrSize){
    int position, j;
    T smallest = arr[K];
    for(int j= K+1; j<arrSize; j++){
        if(arr[j] < smallest){
            smallest = arr[j];
            position = j;
        }
    }
    return position;
}

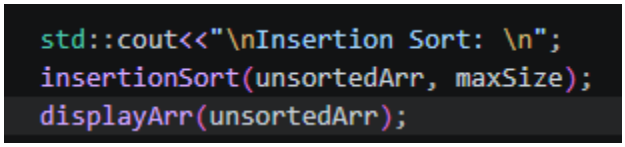
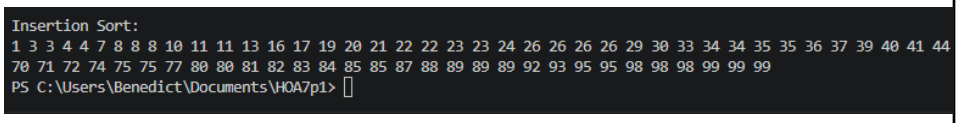
template <typename T>
void selectionSort(T arr[], size_t arrSize){
    int pos, temp, pass;
    for(int i=0; i<arrSize; i++){
        pos = routineSmallest(arr, i, arrSize);
        //swapping
        temp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = temp;
        pass++;
        //for verification and count number of passes
    }
}
```

```
std::cout<<"\nSelection Sort: \n";
selectionSort(unsortedArr, maxSize);
displayArr(unsortedArr);
```

```
Selection Sort:
99 0 2 3 4 5 5 5 6 6 8 11 11 11 15 15 15 16 16 16 17 19 19 21 24 25 28 30 31 31 32 32 34 37
4 75 75 75 75 76 78 80 80 80 81 82 83 83 85 85 86 86 86 86 89 90 92 94 95 97 97 97 99 99
```

Observations

This function has a time complexity with the worst and best case scenario of **O(N^2)**. It is the worst case because if the array is completely reversed or random, every inner loop scan finds a new minimum. While it is the best case scenario because if the array is already sorted, the inner loop still scans everything to confirm.

Code + Console Screenshot	  
Observations	The Worst-Case Scenario in this time complexity is **O(N^2)**, this happens when the array is in reverse order because the outer loop runs n times and the inner loop remains true until J goes all the way down to -1. The Best-Case Scenario is **O(N)** and it happens when the array is already perfectly sorted so the inner loop body never executes.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode
BEGIN DECLARE votes[100] DECLARE candidateNames[5] = {name1, name2, name3, name4, name5} DECLARE voteCounts[5] = {0, 0, 0, 0, 0}

```
// Step 1: Generate random votes
FOR i = 0 TO 99
   votes[i] = RANDOM NUMBER between 1 and 5
END FOR

DISPLAY votes (unsorted)

// Step 2: Sort the votes
CALL insertionSort(votes, 100)
DISPLAY votes (sorted)

// Step 3: Count the votes
FOR j = 0 TO 99
   voteCounts[ votes[j] - 1 ] = voteCounts[ votes[j] - 1 ] + 1
END FOR

// Step 4: Find the candidate with the most votes
bestCount = voteCounts[0]
bestIndex = 0
FOR k = 1 TO 4
   IF voteCounts[k] > bestCount THEN
      bestCount = voteCounts[k]
      bestIndex = k
   END IF
END FOR

// Step 5: Display results
FOR k = 0 TO 4
   DISPLAY "Candidate " + (k+1) + " (" + candidateNames[k] + "): " + voteCounts[k] + " votes"
END FOR

DISPLAY "Winner: " + candidateNames[bestIndex] + " with " + bestCount + " votes"

END
```

Algorithm Codes

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include "sortingAlgo.h"

void displayArr(int arr []);
void countVotes(int arr[], int size);

const int maxSize = 100;

int main () {
   int unsortedArr[maxSize] = {};

    //use time (0) as the seed for random values
    srand(time(0));
    //populate the array with random values from 1-5 for votes
    for (int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%5 + 1;
    }
    //display the randomly generated array
    std::cout<<"Unsorted Votes: \n";
    displayArr(unsortedArr);

    std::cout<<"Votes sorted (Insertion Sort): ";
    insertionSort(unsortedArr, maxSize);
    displayArr(unsortedArr);

    countVotes(unsortedArr, maxSize);
}
```

```cpp
std::string candidateNames[5] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustin",
    "Deja Layla Banaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

//5 for the total number of candidates
void countVotes(int arr[], int size) {
    int voteCounts[5] = {};
    for(int j = 0; j < size; j++) {
        voteCounts[arr[j] - 1]++;
    }

    int bestCount = voteCounts[0];
    int bestIndex = 0;
    for (int k = 1; k < 5; k++) {
        if(voteCounts[k] > bestCount) {
            bestCount = voteCounts[k];
            bestIndex = k;
        }
    }

    //print the tally
    std::cout << "\nVote Tally: \n";
    for(int k = 0; k < 5; k++) {
        std::cout << "Candidate " << (k + 1) << ":" << " (" << candidateNames[k] << "): " << voteCounts[k] << " votes \n";
    }

    std::cout << "\nWinner: Candidate " << (bestIndex + 1) << " " << candidateNames[bestIndex] << " with " << bestCount << " votes\
}
```

Output

```
PS C:\Users\Benedict\Documents\HOA7p1> & C:\Users\Benedict\Documents\HOA7p1\supplementary.exe
Unsorted Votes:
2 1 5 5 3 1 2 5 1 4 4 5 4 4 1 2 3 1 2 5 5 2 2 5 3 5 5 2 2 1 4 1 5 3 5 4 4 5 5 1 5 4 5 5 1 3 4 5 2 2 3 2 4 2 3 5 3 5 5 1 5 1 5 5 4 2 4 5 4 3 5 3 5 5 3 5 3 1 3 5 4 1 3 5 3 3 2 1 1 5 3 2 1 5 1 1 1 5 3 3
Votes sorted (Insertion Sort): 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5
```

elements in the array containing the votes of the candidates provided. **I picked Insertion sort** because my array can only have 5 possible values and that is 1-5 spread across 100 elements. That would mean that there are many duplicate elements that need to travel a long distance to reach their sorted position. Insertion sort depends on how far out of place each element is, with low range between data like this it means that the average element does not move very far so in practice insertion sort does noticeably fewer operations.

Conclusion

In this activity, I managed to learn about the different sorting algorithms that can sort out unstructured elements given. I also learned about how to code these different algorithms and how each one follows a different process to arrange the elements properly. By implementing them, I was able to better understand how sorting works and how these algorithms can be used to organize data efficiently. Overall, this activity helped me improve my understanding of sorting algorithms and my ability to apply them through programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Guidance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Guided me on what to do next after each step in the supplementary activity**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.